

East West University

Department of Computer Science and Engineering

Project Title : Motor-Imagery EEG Classification Using Graph Neural Networks with Functional Channel Connectivity

Course Instructor: Dr. Raihan Ul Islam

Semester : Fall 26 Group No : 03 Section : 02

Name	ID
Nila Ashma Sara	2022-2-60-104
Ipte Salah Rahman Orni	2022-2-60-144
Anika Binte Rashel	2022-3-60-336
Afifa Tabassum Sharly	2022-1-60-311

Submission Date : 21 October 2026

Abstract

This report explains the structure, purpose, data flow, implementation choices, and limitations of the Python notebook EEG_GraphNet.ipynb. The notebook is designed around the BCI Competition IV Dataset 2a and explores four-class motor-imagery EEG data: left hand, right hand, both feet, and tongue. It uses MNE-Python to read GDF recordings and extract epochs, NumPy and pandas for data handling, SciPy for spectral features, and visualization libraries for exploratory data analysis.

The current notebook implements data inspection, epoch preparation, statistical checks, handcrafted feature extraction, dimensionality-reduction visualizations, an interactive signal plot, and CSV exports. Despite the project title, the present code does not yet construct a channel graph or train a Graph Neural Network (GNN). Those steps, along with a controlled evaluation against baseline classifiers, are future implementation tasks. No classification accuracy or experimental outcome is claimed in this report because the notebook does not train and evaluate a classifier.

1. Introduction

Electroencephalography (EEG) measures electrical activity recorded from electrodes placed on the scalp. In motor-imagery Brain–Computer Interface (BCI) research, a participant imagines a movement without necessarily performing it. The EEG signal is then analyzed to infer the intended class.

The notebook studies four motor-imagery classes: left-hand movement, right-hand movement, both-feet movement, and tongue movement. The intended longer-term research direction is to model relationships among EEG channels using a graph-based neural network. The existing code builds the data-analysis foundation needed before a graph model can be implemented.

2. Project Objectives

1. Inspect the dataset files and understand their structure.
2. Read EEG recordings in GDF format and inspect MATLAB MAT files.
3. Extract labeled epochs from continuous EEG recordings.
4. combine trials from training recordings and retain subject/trial metadata.
5. Examine raw signal statistics, possible outliers, invalid values, duplicates, and channel correlations.
6. Check class balance across the full set and across subjects.
7. Extract interpretable per-channel features, including log-variance and relative frequency-band power.
8. Explore feature distributions and correlations.
9. Visualize high-dimensional features using PCA, t-SNE, and UMAP.
10. Save analysis tables to Google Drive for further study.

3. Dataset and Assumptions

The notebook is configured to use the BCI Competition IV Dataset 2a, a motor-imagery EEG dataset with nine participants, four classes, 22 EEG channels, and three EOG channels. The dataset is commonly sampled at 250 Hz and contains two sessions per participant. The expected trial count

before project-specific exclusions is 288 trials per session; this is a dataset expectation, not a verified count from the notebook run.

The configured Google Drive directory is /content/drive/MyDrive/475_mini_proj. The code searches recursively for GDF and MAT files. Training files are identified using a filename pattern such as A01T.gdf through A09T.gdf. The code expects event codes 769, 770, 771, and 772 to represent the four motor-imagery classes; this mapping must be verified against the actual annotations and dataset documentation.

The notebook assumes that recordings selected for concatenation have compatible EEG channel counts and order, sampling frequencies, and epoch lengths. These conditions should be explicitly validated before combining subjects. MAT files are inspected but are not automatically merged with GDF recordings, because their variables, units, labels, and preprocessing history may differ.

4. Software and Libraries

Python is the implementation language, and Google Colab is the expected execution environment. The notebook installs and imports the following main packages:

- MNE-Python: reading GDF files, inspecting annotations, selecting EEG channels, and extracting epochs.
- NumPy: numerical arrays, transformations, statistics, and handling invalid values.
- pandas: metadata, tabular feature matrices, summary statistics, and CSV exports.
- SciPy: loading MAT files and estimating spectral power using Welch's method.
- Matplotlib and Seaborn: static statistical charts and correlation heatmaps.
- Plotly: interactive histograms, boxplots, violin plots, and EEG waveform charts.
- scikit-learn: standardization, PCA, and t-SNE.
- UMAP: nonlinear dimensionality reduction for exploratory visualization.

5. Code-Wise Explanation

5.1. Package installation (Cell 1): The notebook installs MNE, SciPy, scikit-learn, Seaborn, Plotly, and umap-learn. The -q option suppresses much of the installation output. In Colab, installation may need to be rerun when the runtime is reset.

5.2. Imports and setup (Cell 3): The code imports filesystem tools, warning utilities, NumPy, pandas, plotting libraries, SciPy's MAT-file reader, MNE, and Plotly. Later cells import functions for spectral estimation, PCA, t-SNE, and feature scaling.

5.3. Google Drive mount (Cell 5): drive.mount('/content/drive') makes Drive files accessible to the Colab runtime. This cell is Colab-specific and will not work unchanged in a normal local Python environment.

5.4. File discovery (Cells 6–7 and 12): os.walk recursively traverses the configured data folder. Paths ending in .gdf and .mat are collected into separate lists. A file-summary table records file names, extensions, and file sizes. The notebook also attempts to read GDF recording metadata such as sampling rate, duration, and channel names.

5.5. MAT inspection (Cell 10): inspect_mat_file() uses scipy.io.loadmat() and prints variable names, dimensions, and data types while skipping internal MATLAB metadata keys. This is an inspection

step; it does not prove that MAT data is compatible with the GDF data. MATLAB v7.3 files may require an HDF5-compatible reader.

5.6. GDF event inspection (Cell 19): MNE converts annotations into events using events_from_annotations(). The resulting event IDs are inspected to verify whether the expected motor-imagery event codes are present.

5.7. Epoch extraction (function used in Cells 19–20): load_gdf_epochs() reads a GDF recording, identifies the expected class events, selects EEG channels, and segments continuous signals into epochs from 0 to approximately 3.996 seconds after the event. It returns the epoch array, labels, metadata, channel names, and sampling frequency. The function expects the four classes to be available; a missing or differently encoded class can cause a failure.

5.8. Epoch tensor and labels (Cells 20–22): MNE returns epoch data in a three-dimensional array shaped trials × channels × samples. The code obtains event IDs for labels and stores subject/trial/source information in a metadata table. Each training recording is processed, then NumPy concatenates the epoch arrays along the trial axis and pandas combines the metadata. A typical expected shape for one full subject is (288, 22, 1000), but actual dimensions must be checked after execution.

5.9. Raw-channel statistics (Cells 24–29): The epoch tensor is rearranged so each channel becomes a column and its values across trials and time are stacked as observations. The descriptive_statistics() function computes mean, median, standard deviation, minimum, quartiles, maximum, skewness, and excess kurtosis. A histogram with a KDE curve visualizes one channel. The IQR method flags values outside Q1 − 1.5×IQR and Q3 + 1.5×IQR as potential outliers; these values are flagged for inspection, not automatically removed.

5.10. Data-quality checks (Cells 31–35): The notebook counts NaN and infinite signal values. It flattens each epoch to check for exact duplicate signal arrays and checks repeated subject/trial metadata combinations. It also flags near-constant channels using a variance threshold and identifies highly correlated channel pairs using Pearson correlation, with an absolute correlation threshold of 0.90. These checks are screening tools rather than proof that all data-quality issues have been found.

5.11. Class-balance analysis (Cells 37–39): The code counts trials per class and computes their percentages, then displays class balance in a chart. A cross-tabulation shows how many trials of each class occur for each subject. This is useful for detecting missing trials or class imbalance after epoch rejection.

5.12. Feature extraction (Cells 41–44): extract_epoch_features() calculates per-channel log-variance and relative power for theta (4–8 Hz), mu/alpha (8–13 Hz), beta (13–30 Hz), and gamma (30–40 Hz). Welch's method estimates the power spectral density. Band power is approximated by integrating PSD values within a frequency mask; relative power is band power divided by total estimated power. A small epsilon prevents division by zero or taking the logarithm of zero. The resulting feature table has one row per trial and multiple feature columns per channel.

5.13. Feature table and metadata (Cell 44): Feature values are cleaned for positive or negative infinity, then joined to subject/class metadata. This combined table supports class-wise analysis and export. Any remaining missing values need explicit treatment before a downstream classifier is trained.

5.14. Feature distributions (Cells 46–52): The notebook selects a representative feature, usually a mu/alpha relative-power feature from the first EEG channel. Plotly histograms, boxplots, and violin

plots compare the feature distribution across classes. Different distributions may indicate potentially useful features, but visual separation alone is not proof of predictive performance.

5.15. Feature correlation (Cells 54–56): Pearson correlations between feature columns are displayed as a heatmap. Feature pairs with absolute correlation at least 0.90 are listed as potentially redundant. High correlation can indicate overlapping information, but it does not automatically mean one feature should be removed.

5.16. Dimensionality reduction (Cells 59–65): Invalid feature values are replaced or imputed for visualization, and StandardScaler standardizes features. PCA creates two linear components that capture directions of high variance. t-SNE creates a nonlinear two-dimensional embedding and uses a random subset of up to 1,500 trials to reduce computation. UMAP creates another two-dimensional embedding using n_neighbors=15 and min_dist=0.1. These embeddings are exploratory visualizations, not classification scores.

5.17. EEG waveform visualization (Cells 66–67): The notebook selects one trial and one channel, converts sample indices to seconds using the sampling frequency, and plots amplitude in microvolts against time. This can help visually inspect an example signal but does not replace systematic artifact analysis.

5.18. CSV export (Cells 68–69): The code creates an EDA_Results folder under the dataset directory. A safe_to_csv() helper exports tables only when the named variables exist. The intended outputs include file summaries, recording summaries, class balance, channel statistics, outlier summaries, channel/feature correlations, the combined feature table, and PCA/t-SNE/UMAP coordinates. A successful export message does not necessarily mean every optional table was generated; the files should be checked in Drive.

6. Data Flow and Main Variables

The primary data flow is: dataset files → recording inspection → event detection → epoch extraction → combined epoch tensor and labels → data-quality analysis → handcrafted feature extraction → feature-level EDA → dimensionality reduction → CSV exports.

X: three-dimensional EEG epoch tensor with shape (trials, channels, time samples).

y: one label per retained epoch, derived from event codes.

metadata: subject ID, trial information, class name, and source recording information.

raw_channel_df: a channel-wise table used for raw signal statistics.

feature_df: one row per trial and one column per extracted feature.

eda_df: metadata combined with extracted features.

F_scaled: standardized feature matrix used by PCA, t-SNE, and UMAP.

pca_df, tsne_df, umap_df: two-dimensional coordinates with class and subject labels.

7. Mathematical Methods

7.1. Log-variance: for signal samples $x_1,\ldots,x_n$, variance measures amplitude variation. The feature is log(variance + ε), where ε is a small positive constant used to avoid log(0).

7.2. Welch PSD: Welch's method estimates how signal power is distributed across frequency. The code uses the sampling rate and a segment length capped at 256 samples. The resulting frequency resolution depends on the sampling rate and segment length.

7.3. Relative band power: relative_power = band_power / total_power. It describes the fraction of estimated spectral power within a chosen frequency band. The selected bands are theta, mu/alpha, beta, and gamma.

7.4. IQR outlier rule: IQR = Q3 − Q1. Values below Q1 − 1.5×IQR or above Q3 + 1.5×IQR are flagged as potential outliers.

7.5. Standardization: z = (x − mean) / standard deviation. Scaling helps prevent features with larger numerical units from dominating distance-based visualizations.

7.6. Pearson correlation: correlation lies between −1 and +1 and summarizes linear association between two variables. Correlation is not proof of causal or direct neural communication.

8. What Is Not Yet Implemented

The current notebook is primarily a dataset preparation and exploratory data analysis pipeline. It does not yet implement the core GNN classification system described in the project title.

Missing implementation stages include: (1) an explicit graph representation with EEG channels as nodes; (2) a defined edge/adjacency construction method, such as electrode-distance or a carefully documented functional connectivity measure; (3) a graph neural network architecture and training loop; (4) train/validation/test splitting; (5) classifier predictions and evaluation metrics; and (6) comparison with baseline models.

Metrics such as accuracy, macro-F1, balanced accuracy, per-class recall, and a confusion matrix should be calculated only after a model has been trained and evaluated on an appropriate held-out set. The notebook's imports or project title do not mean these results have been produced.

9. Important Limitations and Recommended Improvements

9.1. Verify event mapping: print the annotation mapping for every recording and confirm that the four labels match the official dataset protocol.

9.2. Validate recordings before concatenation: confirm identical channel names/order, channel count, sampling frequency, units, and epoch sample count. Reorder channels consistently or exclude incompatible recordings with a documented reason.

9.3. Confirm signal units: MNE's get_data(units='uV') is intended to return EEG in microvolts. Ensure that the installed MNE version supports this parameter and that the values are not converted twice.

9.4. Review preprocessing: the dataset may already be filtered. Do not apply extra filters automatically; document any additional band-pass/notch filtering and artifact rejection and apply equivalent preprocessing to compared models.

9.5. Improve artifact checks: inspect EOG contamination, unusually large amplitudes, flat channels, and rejected epochs. Statistical outliers should be reviewed rather than removed without justification.

9.6. Validate feature computation: ensure the chosen PSD frequency resolution adequately represents each band. Handle bands with no frequency bins deliberately, and document whether total power is integrated over the entire estimated spectrum or a specified range.

9.7. Avoid leakage in evaluation: for supervised evaluation, fit imputation, scaling, feature selection, and dimensionality reduction using training data only, then apply the fitted transformations to validation/test data. The current full-data scaling is suitable for exploratory visualization but not a leakage-free classifier evaluation pipeline.

9.8. Use subject-aware splits: EEG varies between participants. If the research goal is generalization to unseen people, use subject-wise testing such as leave-one-subject-out or grouped splits. Do not randomly split epochs from the same subject across train and test without explaining the intended evaluation.

9.9. Make random sampling reproducible: keep fixed random seeds and record library versions and the exact dataset files used.

9.10. Check exported files: inspect the EDA_Results directory after execution and verify each expected CSV exists and has the intended row/column counts.

10. Proposed GNN Development Plan

Step 1: Decide graph nodes. A straightforward option is one node per EEG channel.

Step 2: Decide node features. The existing per-channel log-variance and band-power features can form a node-feature vector for each trial.

Step 3: Define graph edges. Compare an electrode-distance graph with a documented functional connectivity graph. A learnable adjacency matrix could be considered if time and data support it. Functional association should not be described as proof of direct causal brain communication.

Step 4: Implement the GNN using a suitable graph-learning library. Define message passing, graph pooling/readout, and a four-class output layer.

Step 5: Establish evaluation splits before fitting transformations or training. Use subject-wise splits if cross-subject generalization is the research goal.

Step 6: Train and tune using training/validation data only. Keep the test set isolated until final evaluation.

Step 7: Compare against appropriate baselines such as CSP-LDA, CSP-SVM, and EEGNet using the same data protocol and clearly documented metrics.

Step 8: Report mean and variability across subjects or folds, confusion matrices, class-wise precision/recall/F1, balanced accuracy, and the exact split protocol. Do not claim an improvement unless the experiments support it.

11. Conclusion

The notebook establishes a useful initial workflow for exploring motor-imagery EEG data. It discovers files, inspects GDF and MAT structures, extracts labeled epochs, performs data-quality checks, calculates interpretable spectral and variance features, visualizes class-wise patterns, and exports analysis results.

Its main contribution at the current stage is data preparation and exploratory analysis rather than a completed predictive model. To match the stated project title, the next major tasks are to define the EEG channel graph, implement and train a GNN, prevent data leakage, and compare the model against established baselines under a reproducible evaluation protocol. Actual dataset counts, plots, and numerical results should be added only after the notebook is executed and verified.